

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – CASE STUDY

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO3: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Case Study
Weightage:	20% of CIA	Total Marks:	100
CO3 Statement:	Design Divide and Conquer algorithms for Merge Sort, Quick Sort, Binary Search and Matrix Multiplication		
Student Name:	Akshaya A	Reg. No.:	192421222
Section / Batch:	SLOT B	Date:	05/08/2026

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given case study questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues, provide an in-depth analysis, and propose innovative, feasible solutions with strong justification.

Question Type	Focus Area	Questions
Case Study	Focus on Design Divide and Conquer algorithms: Merge Sort, Quick Sort, Binary Search and Matrix Multiplication	Q1

SECTION A – CASE STUDY

Code of Conduct

I Akshaya A/192421222(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Akshaya A

RUBRICS FOR CALCULATION

Case Study					
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25%	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20%	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20%	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10%	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25%				
Application of Concepts	25%				
Analysis	20%				
Solution Design	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

NAME: AKSHAYA A

REG NO:192421222

COURSE: CSA0603 DESIGN AND ANALYSIS OF ALGORITHM

CO3_AT2_CASE STUDY

TITLE

REAL-TIME RANKING SYSTEM USING QUICK SORT

1. Description of the Real-World Problem

In today's digital world, many applications require data to be sorted instantly as new information becomes available. Examples include online gaming leaderboards, coding competition rankings, sports tournaments, and educational platforms. These systems continuously receive updated scores from users, making it necessary to rearrange the rankings in real time.

A ranking system must be capable of sorting large amounts of data quickly while maintaining accuracy. Slow sorting algorithms can delay leaderboard updates, affecting user experience and system performance. Therefore, selecting an efficient sorting algorithm is essential for ensuring fast and reliable ranking updates.

Quick Sort is one of the most widely used sorting algorithms because it provides excellent average-case performance. It follows the **Divide and Conquer** strategy by dividing the dataset into smaller partitions, sorting them independently, and combining the results to produce a

fully sorted list. This makes Quick Sort highly suitable for real-time ranking systems where speed and efficiency are important.

2. Problem Statement

Design and analyze a real-time ranking system that sorts player scores using the **Quick Sort algorithm**. The system should efficiently update the leaderboard whenever player scores change. The analysis should include the working of Quick Sort using the Divide and Conquer approach, the impact of pivot selection, time complexity under different cases, and an optimized solution using randomized pivot selection to improve performance and reduce the chances of worst-case execution.

3. Algorithm Used

Quick Sort Algorithm

Quick Sort is a comparison-based sorting algorithm that uses the Divide and Conquer technique. It works by selecting one element as a pivot and partitioning the remaining elements into two groups. Elements smaller than the pivot are placed on the left, while elements larger than the pivot are placed on the right. The same process is then repeated recursively for both partitions until the entire array is sorted.

Algorithm Steps

1. Select a pivot element from the array.
2. Compare every element with the pivot.
3. Place smaller elements to the left of the pivot.
4. Place larger elements to the right of the pivot.
5. Recursively apply the same process to the left and right partitions.
6. Continue until all elements are sorted.

4. Justification for Choosing the Algorithm (Why Quick Sort?)

Quick Sort is selected because it offers excellent performance for sorting large datasets and is widely used in practical applications. Its average-case time complexity of $O(n \log n)$ makes it significantly faster than simple sorting algorithms such as Bubble Sort and Selection Sort.

Another advantage is that Quick Sort performs **in-place sorting**, meaning it requires very little additional memory. This makes it suitable for systems where memory efficiency is important. Since real-time ranking systems constantly receive updated scores, Quick Sort can efficiently rearrange the leaderboard without excessive computational cost.

To further improve performance, randomized pivot selection can be used. This reduces the possibility of worst-case execution and helps maintain balanced partitions during sorting.

5. Complexity Analysis

Time Complexity

Case	Complexity
------	------------

Best Case	$O(n \log n)$
-----------	---------------

Average Case	$O(n \log n)$
--------------	---------------

Worst Case	$O(n^2)$
------------	----------

Space Complexity

$O(\log n)$ due to recursive function calls.

Analysis

- **Best Case:** Occurs when the pivot divides the array into two nearly equal halves.
- **Average Case:** Random input generally produces balanced partitions, resulting in efficient performance.
- **Worst Case:** Happens when the pivot is always the smallest or largest element, creating highly unbalanced partitions and increasing execution time.

6. Manual Execution (Step-by-Step Process)

Example Input

Player Scores:

80, 55, 95, 70, 60

Step 1

Select **60** as the pivot.

Original Array:

80 55 95 70 60

After partitioning:

55 | 60 | 80 95 70

Step 2

Sort the right partition.

Sub-array:

80 95 70

Select pivot = 70

After partitioning:

70 | 80 95

Step 3

Sort the remaining partition.

Sub-array:

80 95

Select pivot = 95

Sorted Sub-array:

80 95

Final Sorted Scores

55 60 70 80 95

Leaderboard

Rank Score

1 95

2 80

3 70

4 60

5 55

This process is repeated whenever a player's score changes, ensuring that the leaderboard always displays the latest rankings.

7. Applications

Quick Sort is widely used in various real-world applications, including:

- Online gaming leaderboards
- Coding competition ranking systems
- Sports tournament scoreboards
- Student result processing systems
- Search engine result sorting
- Database record management
- Financial and stock market analysis
- E-commerce product ranking systems

8. Advantages

- Fast average-case performance.
- Efficient for large datasets.
- Uses the Divide and Conquer strategy.
- Requires less additional memory.
- Performs in-place sorting.
- Suitable for dynamic ranking systems.
- Easy to implement using recursion.

9. Limitations

Although Quick Sort is efficient, it has a few limitations:

- Worst-case time complexity is **$O(n^2)$** .
- Performance depends on the selection of the pivot.
- Recursive implementation may increase stack memory usage.
- It is not a stable sorting algorithm.
- Poor pivot selection can reduce overall efficiency.

10. Comparison with Other Algorithms

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$

Algorithm	Best Case	Average Case	Worst Case	Space Complexity
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Quick Sort generally performs better than Bubble Sort, Selection Sort, and Insertion Sort when handling large datasets. Compared to Merge Sort, it usually requires less memory and often executes faster in practical applications, although Merge Sort guarantees better worst-case performance.

11. Optimized Solution

One major challenge in Quick Sort is selecting an appropriate pivot element. If the first or last element is always chosen as the pivot, an already sorted array may result in highly unbalanced partitions, leading to the worst-case time complexity of $O(n^2)$.

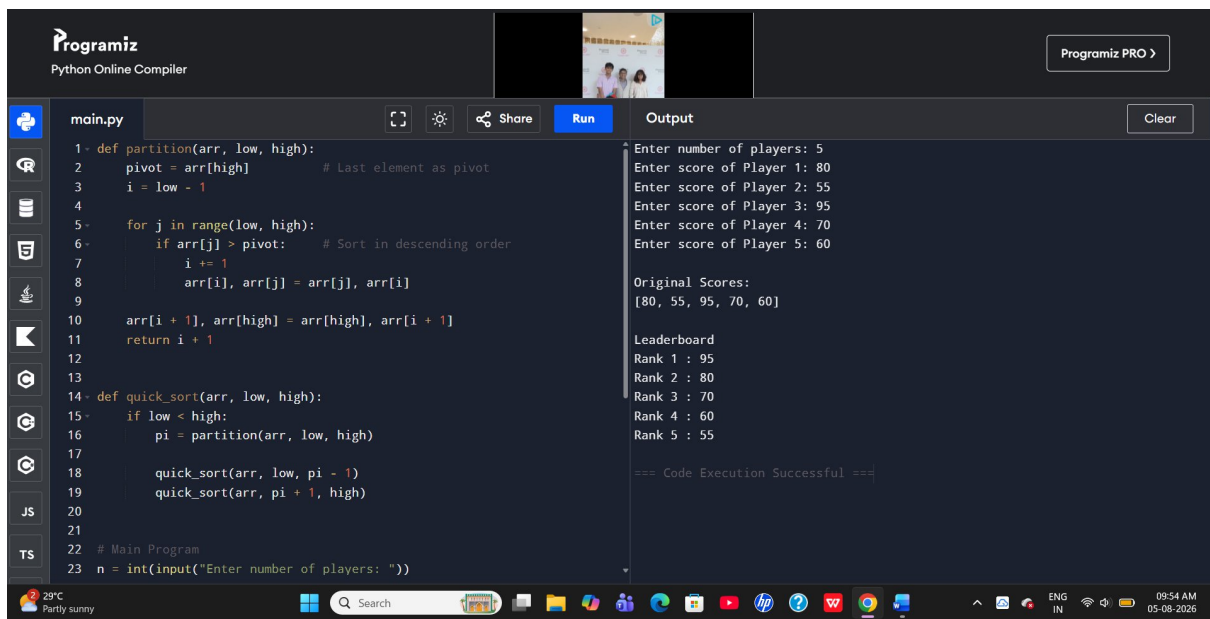
To overcome this issue, **Randomized Pivot Selection** is recommended. In this approach, the pivot is selected randomly before partitioning the array. Randomization significantly reduces the probability of unbalanced partitions and improves the overall efficiency of the algorithm.

Using randomized pivot selection provides the following benefits:

- Reduces the likelihood of worst-case performance.
- Produces more balanced partitions.

- Improves execution speed.
- Maintains average-case complexity of $O(n \log n)$.
- Makes the ranking system more reliable for real-time applications.

12. Screenshot of the Program



The screenshot displays the Programiz Python Online Compiler interface. The code editor on the left contains a Python script for a Quick Sort algorithm. The output panel on the right shows the program's execution results, including user input for the number of players and their scores, the original scores list, and the resulting leaderboard.

```
main.py
1- def partition(arr, low, high):
2-     pivot = arr[high] # Last element as pivot
3-     i = low - 1
4-
5-     for j in range(low, high):
6-         if arr[j] > pivot: # Sort in descending order
7-             i += 1
8-             arr[i], arr[j] = arr[j], arr[i]
9-
10-    arr[i + 1], arr[high] = arr[high], arr[i + 1]
11-    return i + 1
12-
13-
14- def quick_sort(arr, low, high):
15-     if low < high:
16-         pi = partition(arr, low, high)
17-
18-         quick_sort(arr, low, pi - 1)
19-         quick_sort(arr, pi + 1, high)
20-
21-
22- # Main Program
23- n = int(input("Enter number of players: "))
```

Output

```
Enter number of players: 5
Enter score of Player 1: 80
Enter score of Player 2: 55
Enter score of Player 3: 95
Enter score of Player 4: 70
Enter score of Player 5: 60

Original Scores:
[80, 55, 95, 70, 60]

Leaderboard
Rank 1 : 95
Rank 2 : 80
Rank 3 : 70
Rank 4 : 60
Rank 5 : 55

=== Code Execution Successful ===
```

29°C Partly sunny 09:54 AM 05-08-2026

Sample Output

Original Scores:

80 55 95 70 60

Sorted Scores:

55 60 70 80 95

Leaderboard

Rank 1 : 95

Rank 2 : 80

Rank 3 : 70

Rank 4 : 60

Rank 5 : 55

14. Conclusion

Quick Sort is an efficient and practical sorting algorithm for real-time ranking systems. Its Divide and Conquer approach enables fast sorting of dynamically changing data, making it suitable for applications such as online gaming, coding contests, sports rankings, and educational platforms. Although Quick Sort may experience poor performance in the worst case due to improper pivot selection, this issue can be minimized through randomized pivot selection. By combining high average-case performance with low memory usage, Quick Sort remains

one of the best choices for implementing fast and reliable real-time leaderboard systems.